

Міністерство освіти і науки України
Сумський державний університет
Кафедра комп'ютерних наук

Пояснювальна записка

до курсової роботи

з дисципліни

«Програмування»

Викладач – Авраменко Віктор Васильович

Студент - Дудченко Андрій Євгенійович

Група – КН-51/1

Варіант - 14

Суми – 2025

Зміст із зазначенням сторінок

Зміст із зазначенням сторінок	2
1. Постановка задачі.....	3
2. Теоретичний матеріал із теми.	5
3. Опис структури даних та вимоги до них.	8
4. Алгоритм роботи програми	9
5. Опис функцій користувача	15
6. Опис файлів та їх призначення.....	17
7. Список використаних бібліотек.	18
8. Інструкція для роботи з програмою	20
9. Приклад тестування та результати роботи програми.....	22
10. Графіки.....	24
Висновки.....	30
Список використаної літератури.....	32
Додаток А. Програмний код.....	33

1. Постановка задачі

Описати масив структур із 3-х елементів. Кожна структура об'єднує дані для одного варіанту розрахунку.

Необхідно для кожного варіанту на відрізку часу від 0 до T з кроком Δt побудувати графік зміни тягового зусилля, що розвивається електромагнітом постійного струму. Тягове посилення обчислюється за формулою.

$$F = 10^{-5} (0,4\pi I w)^2 S_c / (8\pi l_b^2),$$

де I – струм в обмотці електромагніту (А) ;

w – число витків в обмотці;

S_c – поперечний переріз магніту–проводу;

l_b – повітряний зазор між сердечником та якорем.

Струм в обмотці і повітряний зазор змінюються в часі :

$$I = \begin{cases} i_0(1+kt) \text{ для } t \in [0, \frac{T}{8}] \\ i_0(1+k\frac{T}{8}) \text{ для } t \in [\frac{T}{8}, \frac{3T}{8}] \\ i_0(1+k\frac{T}{8}-k(t-\frac{3T}{8})) \text{ для } t \in [\frac{3T}{8}, \frac{5T}{8}] \\ i_0(1-k\frac{T}{8}) \text{ для } t \in [\frac{5T}{8}, \frac{7T}{8}] \\ i_0(1-k\frac{T}{8}+k(t-\frac{7T}{8})) \text{ для } t \in [\frac{7T}{8}, T] \end{cases}$$

$$k = \begin{cases} k_0(1-e^{-mt}) \text{ для } t \in [0, \frac{T}{2}] \\ k_0(1-e^{-m\frac{T}{2}}) \text{ для } t \in [\frac{T}{2}, T] \end{cases}$$

Тут i_0, k_0, m – задані константи.

$$l = \begin{cases} l_0(1+nt) \text{ для } t \in [0, \frac{T}{4}] \\ l_0(1+n\frac{T}{4}) \text{ для } t \in [\frac{T}{4}, T] \end{cases}$$

l_0, n – задані константи.

Вхідні дані зчитуються з файлу.

Результати обчислень занести в інший файл. Передбачити окремі функції для обчислень I, k, l_b .

Вхідні дані:

1. $T = 100$ с, $\Delta t = 5$ с, $S_c = 10 \text{ см}^2$, $i_0 = 0,1$ А, $k_0 = 0,005$, $m = 0,01$ Гц, $i_0 = 0,1$ А, $n = 0,01$, $w = 1000$.

2. $S_c = 12 \text{ см}^2$, $w = 2000$, $i_0 = 0,2$ А . Решту даних див. пункт 1.

3. $S_c = 15 \text{ см}^2$, $w = 3000$, $i_0 = 0,3$ А . Решту даних див. пункт 1.

2. Теоретичний матеріал із теми.

2.1. Мови програмування C та C++

Хоча в навчальній літературі часто зустрічається спільне позначення C/C++, важливо розуміти, що це дві різні мови, які мають спільну історію:

- **Мова C:** Була створена у 1972 році Деннісом Рітчі. У 1978 році вийшла знаменита праця Б. Кернігана та Д. Рітчі «Мова програмування C», яка тривалий час була неофіційним стандартом. Офіційний стандарт ANSI C (C89) з'явився лише у 1989 році.
- **Мова C++:** Вважається майже «надмножиною» мови C. Це означає, що більшість програм, написаних на C, можуть бути скомпільовані в середовищі C++ без значних змін.

Ключові особливості мови C++:

Об'єктно-орієнтоване програмування (ООП):

На відміну від C, підтримує класи та об'єкти, що дозволяє створювати модульний код для повторного використання та полегшує розробку складних проєктів.

Висока продуктивність:

Надає програмістам високий рівень контролю над системними ресурсами та пам'яттю, що забезпечує максимальну швидкість виконання програм.

Кросплатформеність:

Програми можна адаптувати під різні операційні системи (Windows, macOS, Linux) з мінімальними змінами у коді.

Універсальність:

Використовується для створення потужних ігрових двигунів, браузерів, операційних систем та додатків з графічним інтерфейсом.

Сумісність із C:

Оскільки C++ є розширенням мови C, вона дозволяє використовувати більшість бібліотек та синтаксичних конструкцій свого попередника.

2.2. Інтегроване середовище розробки (IDE)

Для написання та запуску програм використовують IDE (Integrated Development Environment). Це «швейцарський ніж» програміста, який містить:

1. **Текстовий редактор** — для написання коду.
2. **Компілятор** — перетворює зрозумілий людині код у зрозумілі процесору нулі та одиниці (машинний код).
3. **Компоновщи** — збирає окремі частини коду та готові бібліотеки в один файл «.exe».
4. **Бібліотеки** — набори готових функцій наприклад, для математичних обчислень або виведення тексту.

Для навчання в СумДУ базовим середовищем є Microsoft Visual Studio. Оскільки сучасні версії Visual Studio вважають деякі старі функції мови C «небезпечними», на початку програми рекомендується додавати такі директиви:

```
#define _CRT_SECURE_NO_WARNINGS  
#pragma warning(disable:4996)
```

2.3. Алфавіт та ідентифікатори

Кожна програма будується з обмеженого набору символів:

- **Літери:** латинські A-Z, a-z та символ підкреслення _.
- **Цифри:** від 0 до 9.
- **Спеціальні символи:** наприклад: + - * / \ () [] { } < > = (символ пробілу) ? ! : ; . , ' " % & ~ ^ | - всього 29 спеціальних символів..

Ідентифікатори — це імена, які ми даємо змінним або функціям.

Правила створення імен:

1. Можна використовувати літери, цифри та _.
2. Не можна починати ім'я з цифри (тільки з літери або підкреслення).
3. Регістр має значення: MyVar, myvar та MYVAR — це три різні змінні.
4. Довжина ідентифікатора зазвичай не повинна перевищувати 32 символи.

2.4. Типи даних

Тип даних вказує комп'ютеру, скільки пам'яті виділити та які операції можна робити з цим значенням.

Тип	Опис	Розмір
char	Один символ	1 байт
int	Цілі числа	4 байти
float	Дійсні числа	4 байти
double	Дійсні числа з подвійною точністю	8 байтів
long	Цілі числа великого діапазону	4-8 байтів

2.5. Константи та ключові слова

Константи — це значення, які ми записуємо в коді безпосередньо і які не змінюються під час роботи.

- **Цілі:** десяткові (10), восьмирічні (починаються з нуля: 012), шістнадцяткові (починаються з 0x: 0xFF).
- **Символьні:** пишуться в одинарних лапках ('A').
- **Рядкові:** тексти в подвійних лапках ("Hello World").
- **Змінні:** спеціальні команди всередині тексту, наприклад \n — перехід на новий рядок.

Ключові слова — це зарезервовані мовою слова, які не можна використовувати як імена для своїх змінних (наприклад: if, else, while, return, int, void).

2.6. Блок-схеми

Овал – вхід в алгоритм (початок), вихід з алгоритму (кінець).

Прямокутник – опис дії. Процес: обробка (обчислення) даних; дію – присвоєння.

Ромб – логічний блок передачі управління. Перевірка логічної умови – вибір напрямку виконання алгоритму в залежності від деякої умови.

Паралелограм – введення/виведення інформації.

3. Опис структури даних та вимоги до них.

Ім'я параметра у формулі	Змінна у програмі	Тип змінної	Призначення
T	T	double	Час моделювання(с), $T>0$
Δt	dt	double	Крок часу
S_c	Sc	double	Поперечний переріз магнітопроводу(см ²), $S_c>0$
i_0	i0	double	Базовий струм в обмотці(А)
k_0	k0	double	Коефіцієнт для $k(t)$
m	m	double	Показник експоненти, Гц
n	n	double	Коефіцієнт для $l_b(t)$
w	w	double	Число витків в обмотці
l_0	l0	double	Початковий повітряний зазор

Таблиця 1 – структура вхідних даних

4. Алгоритм роботи програми

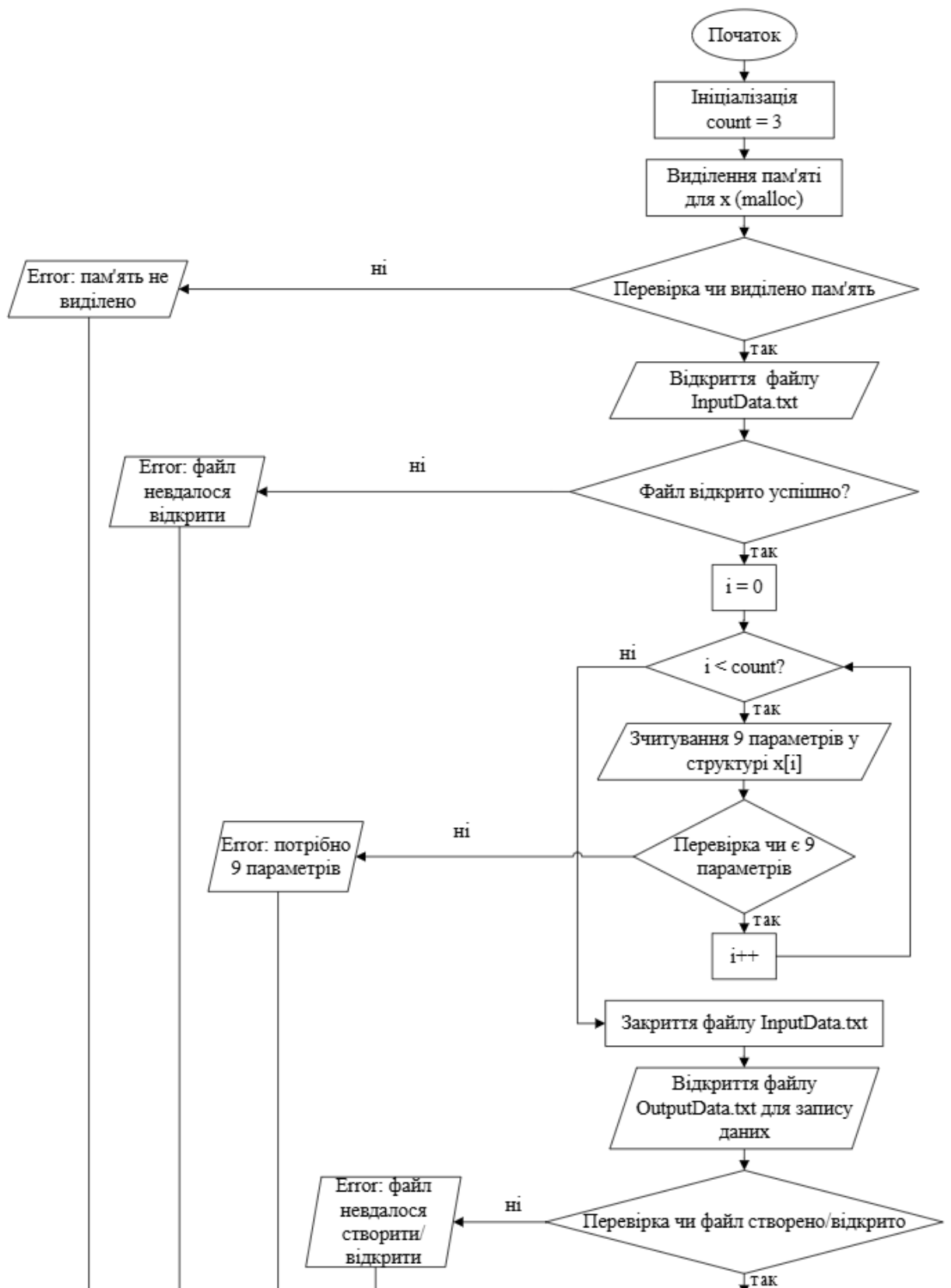


Рисунок 1.1 -Блок-схема (частина 1) алгоритму основної програми (main)

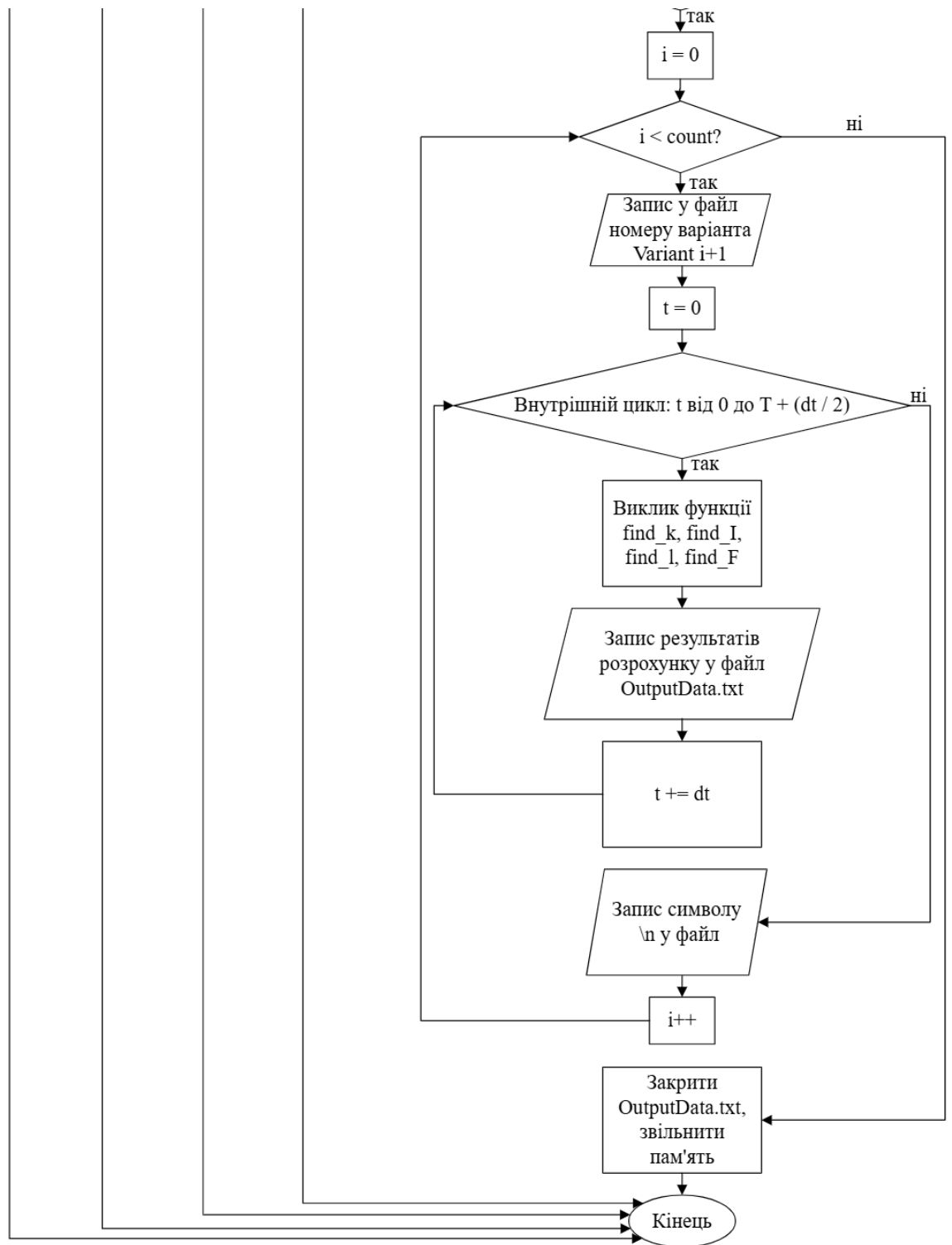


Рисунок 1.2 -Блок-схема (частина 2) алгоритму основної програми (main)

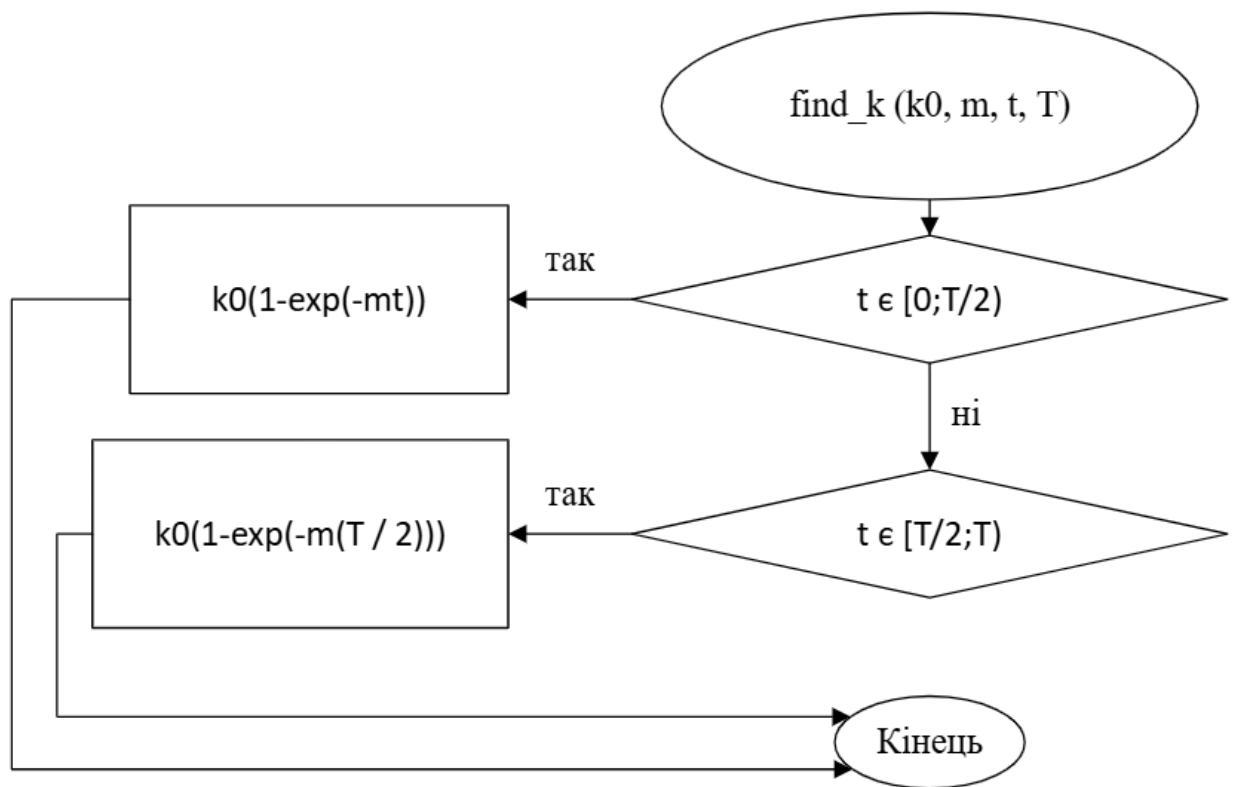


Рисунок 2 - Блок-схема: алгоритм функції find_k для розрахунку коефіцієнта k

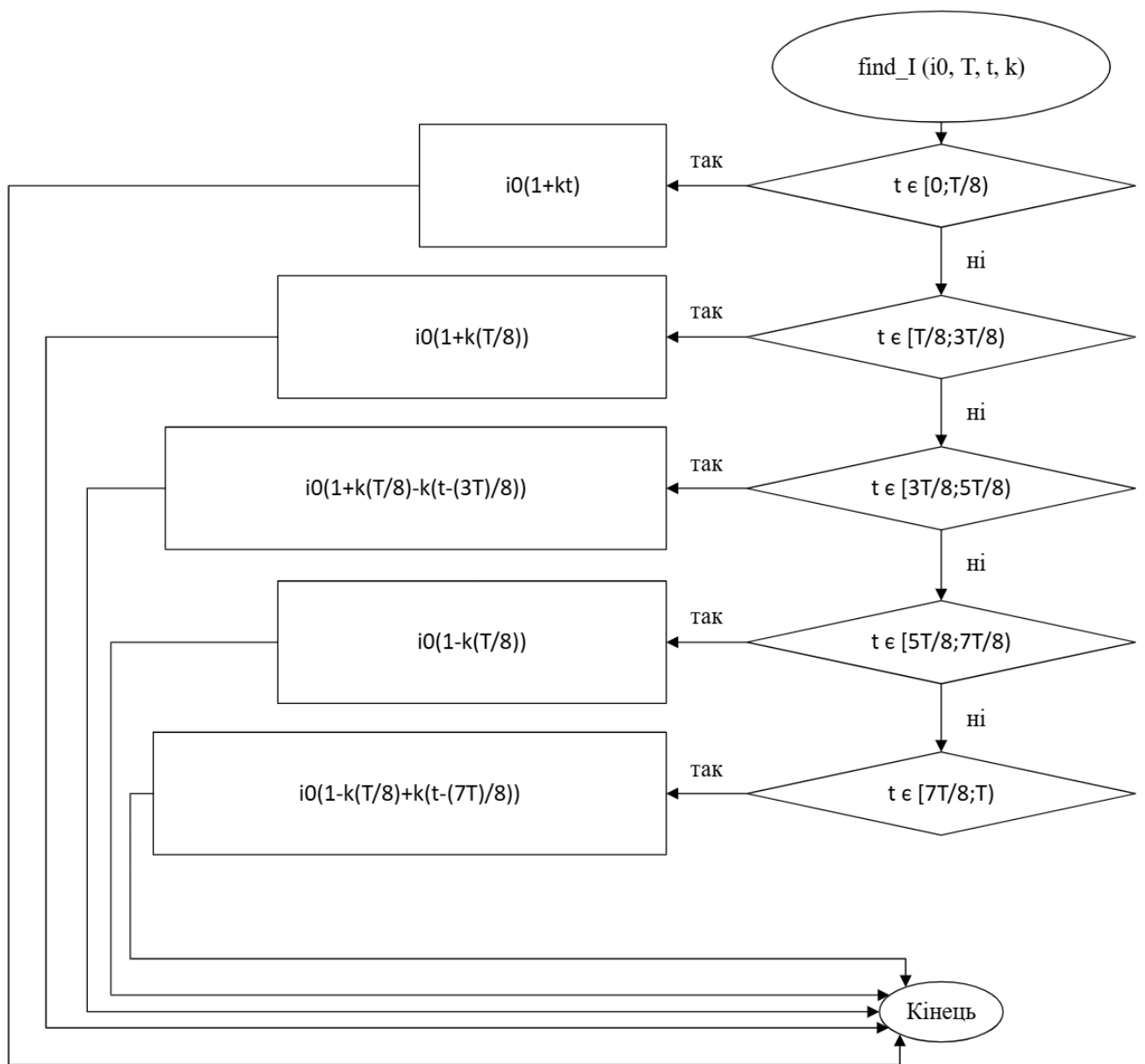


Рисунок 3 - Блок-схема: алгоритм функції find_I для розрахунку сили струму I

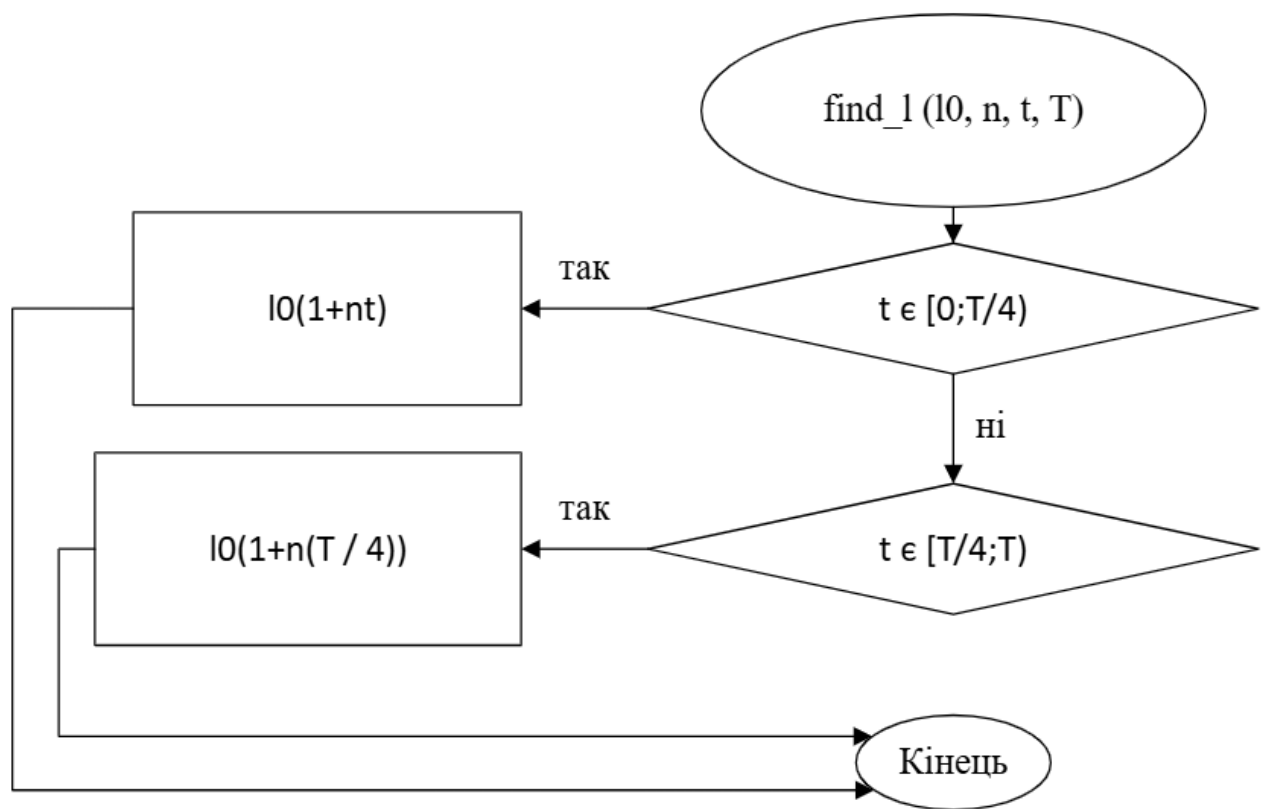


Рисунок 4 - Блок-схема: алгоритм функції `find_1` для визначення зазору `l`

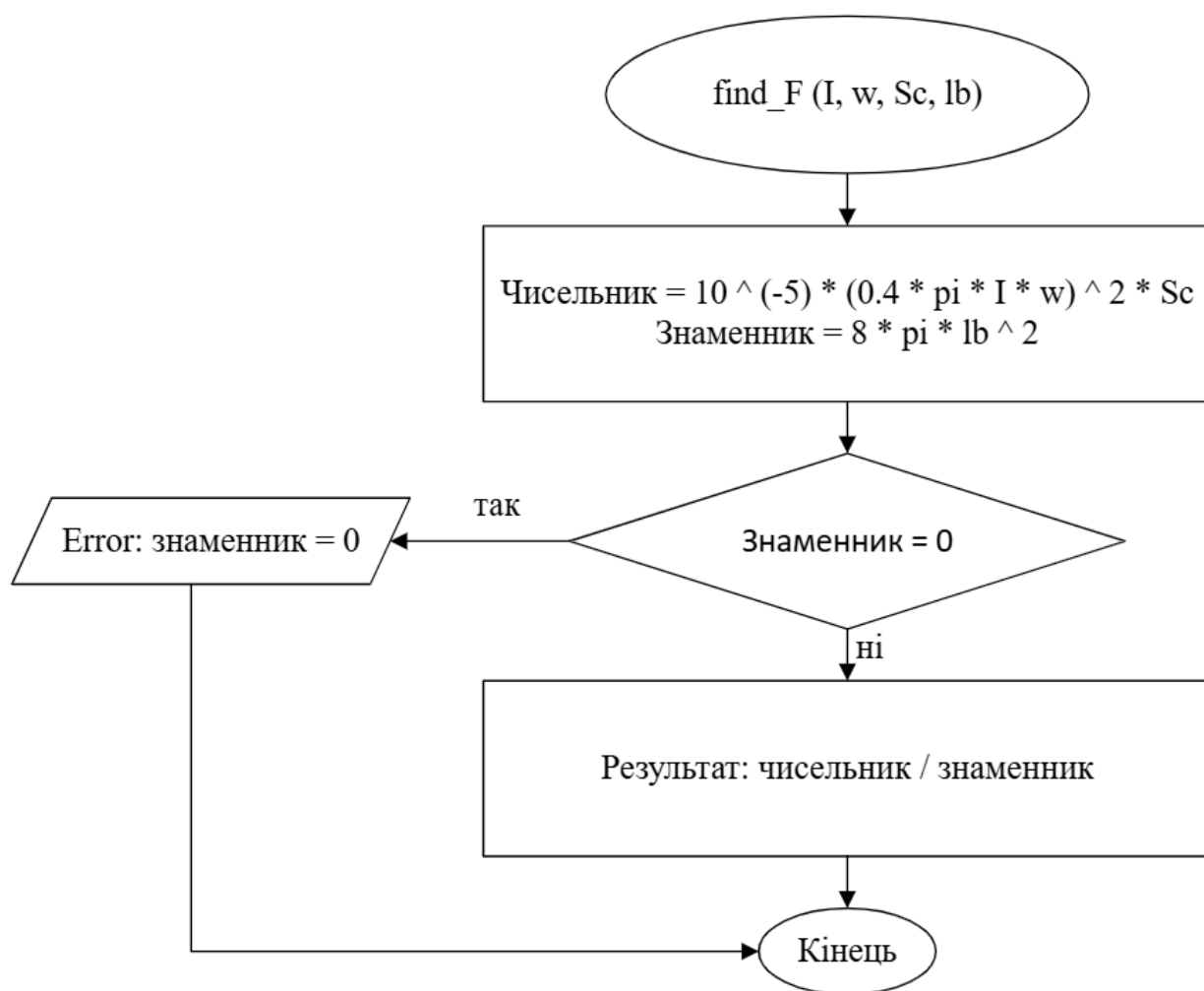


Рисунок 5 - Блок-схема: алгоритм функції `find_F` для обчислення тягового зусилля F

5. Опис функцій користувача

У програмі реалізовано чотири основні обчислювальні функції, кожна з яких відповідає за певну частину математичної моделі електромагніту.

1. **double find_k(double k0, double m, double t, double T)**

Призначення: обчислює значення коефіцієнта k на основі експоненціальної залежності від часу.

Математична модель:

- якщо $t \in \left[0; \frac{T}{2}\right)$, використовується формула: $k = k_0(1 - e^{-mt})$.
- якщо $t \in \left[\frac{T}{2}; T\right]$, використовується формула: $k = k_0 \left(1 - e^{-m\frac{T}{2}}\right)$.

2. **double find_I(double i0, double T, double t, double k_num)**

Призначення: визначає часову залежність сили струму за п'ятьма характерними інтервалами.

Особливість: Функція приймає розраховане значення коефіцієнта k як вхідний аргумент для визначення динаміки струму.

Математична модель:

- якщо $t \in \left[0; \frac{T}{8}\right)$, використовується формула: $I = i_0(1 + kt)$.
- якщо $t \in \left[\frac{T}{8}; \frac{3T}{8}\right)$, використовується формула: $I = i_0 \left(1 + k\frac{T}{8}\right)$.
- якщо $t \in \left[\frac{3T}{8}; \frac{5T}{8}\right)$, використовується формула: $I = i_0 \left(1 + k\frac{T}{8} - k\left(t - \frac{3T}{8}\right)\right)$.
- якщо $t \in \left[\frac{5T}{8}; \frac{7T}{8}\right)$, використовується формула: $I = i_0 \left(1 - k\frac{T}{8}\right)$.
- якщо $t \in \left[\frac{7T}{8}; T\right]$, використовується формула: $I = i_0 \left(1 - k\frac{T}{8} + k\left(t - \frac{7T}{8}\right)\right)$.

3. **double find_l(double l0, double n, double t, double T)**

Призначення: обчислює зміну відстані між осердям та якорем електромагніту.

Математична модель:

- якщо $t \in \left[0; \frac{T}{4}\right)$, використовується формула: $l = l_0(1 + nt)$.
- якщо $t \in \left[\frac{T}{4}; T\right]$, використовується формула: $l = l_0 \left(1 + n \frac{T}{4}\right)$.

4. **double find_F(double I, double w, double Sc, double lb)**

Призначення: виконує фінальний розрахунок механічної сили, що розвивається електромагнітом.

Особливість: Включає захист від критичної помилки — перевірку знаменника на нуль перед виконанням ділення.

$$\text{Формула: } F = 10^{-5} \cdot \frac{(0.4 \cdot \pi \cdot I \cdot w)^2 \cdot S_c}{8\pi l_b^2}.$$

6. Опис файлів та їх призначення

У цій програмі використовуються три основні типи файлів:

1. Курсова робота.cpp:

Тип: вихідний код на мові C++.

Призначення: реалізація логіки математичного моделювання електромагніту.

Опис: містить опис структури InputData, функції для розрахунку фізичних параметрів (k , I , l , F) та головну функцію main, яка керує потоком даних, а саме зчитування, обчислення у циклі, запис.

2. InputData.txt:

Тип: текстовий файл даних.

Призначення: зберігання вхідних параметрів для трьох варіантів розрахунку.

Опис: кожен рядок містить 9 значень: T , Δt , w , S_c , i_0 , k_0 , l_0 , m , n . Програма зчитує ці дані для ініціалізації масиву структур.

3. OutputData.txt:

Тип: текстовий файл результатів.

Призначення: записування результатів розрахунку програми.

Опис: створюється програмою автоматично. У нього записуються результати обчислень у вигляді таблиці для кожного варіанта: час (t), сила струму (I), коефіцієнт (k), повітряний зазор (l) та тягове зусилля (F).

7. Список використаних бібліотек.

Програма використовує стандартні бібліотеки мови C для забезпечення базового функціоналу:

1. `_CRT_SECURE_NO_WARNINGS`

Вона вказує компілятору ігнорувати попередження про використання класичних функцій мови C (таких як `fopen` та `fscanf`), які вважаються потенційно небезпечними в сучасних стандартах, але є необхідними для забезпечення сумісності та низькорівневої роботи з даними.

2. `<stdio.h>`:

Використовується для роботи з файлами (`fopen`, `fprintf`, `scanf`, `printf`) та виводу повідомлень про помилки в консоль (`printf`).

Робота з файлами: за допомогою функцій `fopen` та `fclose` реалізовано безпечне відкриття файлу вхідних параметрів `InputData.txt` та створення звіту `OutputData.txt`.

Зчитування даних: функція `fscanf` виконує форматоване зчитування 9 фізичних параметрів для кожного варіанта розрахунку.

Запис результатів: функція `fprintf` дозволяє структурувати результати обчислень (t, I, k, l, F) у вигляді таблиці для подальшої обробки в Excel.

Контроль помилок: через функцію `printf` програма інформує про критичні ситуації, такі як відсутність файлів або помилка вхідних даних.

3. `<stdlib.h>`:

Використовується для ефективного керування оперативною пам'яттю комп'ютера під час виконання програми.

Динамічне виділення: функція `malloc` виділяє блок пам'яті під масив структур `InputData`, що дозволяє програмі обробляти декілька варіантів розрахунку одночасно.

Звільнення ресурсів: Після завершення циклів обчислення функція `free` повертає виділену пам'ять операційній системі, запобігаючи її витоку.

4. `<math.h>`:

Надає інструменти для реалізації складних фізичних залежностей, закладених у математичну модель електромагніту.

Обчислення експоненти: Функція `exp()` є критичною для роботи підпрограми `find_k`. Вона дозволяє розрахувати динаміку зміни коефіцієнта k .

8. Інструкція для роботи з програмою

Для забезпечення коректної роботи програми та отримання точних результатів моделювання електромагніту, необхідно дотримуватися наступного алгоритму дій:

Крок 1: Підготовка середовища та вхідних даних

1. Розміщення файлів: Переконайтеся, що файл із вихідним кодом Курсова `робота.cpp` та текстовий файл вхідних даних `InputData.txt` знаходяться в одній кореневій директорії проєкту.
2. Форматування вхідних даних: Відкрийте `InputData.txt` та внесіть параметри для трьох варіантів розрахунку. Кожен рядок має містити рівно 9 числових значень, розділених пробілом:
 - Порядок параметрів: T , Δt , w , S_c , i_0 , k_0 , l_0 , m , n .
 - Важливо: Використовуйте крапку як десятковий розділювач
 - Приклад рядка: 100 5 1000 10 0.1 0.005 0.1 0.01 0.01.

Крок 2: Компіляція та запуск програми

1. Запуск IDE: Відкрийте файл `Курсова робота.cpp` у середовищі розробки Microsoft Visual Studio.
2. Виконання: Запустіть програму без налагодження, натиснувши `Ctrl+F5`.

Крок 3: Моніторинг процесу та обробка помилок

Під час виконання звертайте увагу на повідомлення в консолі. Програма реалізує наступну систему діагностики:

- "Error: memory" — недостатньо оперативної пам'яті для виділення масиву структур.
- "Error: cannot open InputData.txt" — файл вхідних даних відсутній або пошкоджений.
- "Error: you need 9 values in ... variant" — у вказаному варіанті бракує даних для ініціалізації структури.

- "Error: denominator = 0" — виявлено спробу ділення на нуль при розрахунку сили F .

Крок 4: Отримання та аналіз результатів

1. Після завершення роботи перейдіть у папку проєкту.
2. Відкрийте автоматично створений файл OutputData.txt.
3. Результати представлені у вигляді структурованих блоків для кожного варіанта. Кожен рядок містить розраховані значення t , I , k , l , F .

9. Приклад тестування та результати роботи програми

Курсова робота.cpp		InputData.txt		OutputData.txt	
1	Variant 1:				
2	t = 0.000000	I = 0.100000	k = 0.000000	l = 0.100000	F = 6.283185
3	t = 5.000000	I = 0.100122	k = 0.000244	l = 0.105000	F = 5.712940
4	t = 10.000000	I = 0.100476	k = 0.000476	l = 0.110000	F = 5.242248
5	t = 15.000000	I = 0.100871	k = 0.000696	l = 0.115000	F = 4.834073
6	t = 20.000000	I = 0.101133	k = 0.000906	l = 0.120000	F = 4.462750
7	t = 25.000000	I = 0.101382	k = 0.001106	l = 0.125000	F = 4.133194
8	t = 30.000000	I = 0.101620	k = 0.001296	l = 0.125000	F = 4.152573
9	t = 35.000000	I = 0.101846	k = 0.001477	l = 0.125000	F = 4.171048
10	t = 40.000000	I = 0.101648	k = 0.001648	l = 0.125000	F = 4.154903
11	t = 45.000000	I = 0.100906	k = 0.001812	l = 0.125000	F = 4.094428
12	t = 50.000000	I = 0.100000	k = 0.001967	l = 0.125000	F = 4.021239
13	t = 55.000000	I = 0.099016	k = 0.001967	l = 0.125000	F = 3.942516
14	t = 60.000000	I = 0.098033	k = 0.001967	l = 0.125000	F = 3.864572
15	t = 65.000000	I = 0.097541	k = 0.001967	l = 0.125000	F = 3.825891
16	t = 70.000000	I = 0.097541	k = 0.001967	l = 0.125000	F = 3.825891
17	t = 75.000000	I = 0.097541	k = 0.001967	l = 0.125000	F = 3.825891
18	t = 80.000000	I = 0.097541	k = 0.001967	l = 0.125000	F = 3.825891
19	t = 85.000000	I = 0.097541	k = 0.001967	l = 0.125000	F = 3.825891
20	t = 90.000000	I = 0.098033	k = 0.001967	l = 0.125000	F = 3.864572
21	t = 95.000000	I = 0.099016	k = 0.001967	l = 0.125000	F = 3.942516
22	t = 100.000000	I = 0.100000	k = 0.001967	l = 0.125000	F = 4.021239
23					
24	Variant 2:				
25	t = 0.000000	I = 0.200000	k = 0.000000	l = 0.100000	F = 120.637158
26	t = 5.000000	I = 0.200244	k = 0.000244	l = 0.105000	F = 109.688448
27	t = 10.000000	I = 0.200952	k = 0.000476	l = 0.110000	F = 100.651160
28	t = 15.000000	I = 0.201741	k = 0.000696	l = 0.115000	F = 92.814197
29	t = 20.000000	I = 0.202266	k = 0.000906	l = 0.120000	F = 85.684804
30	t = 25.000000	I = 0.202765	k = 0.001106	l = 0.125000	F = 79.357325
31	t = 30.000000	I = 0.203240	k = 0.001296	l = 0.125000	F = 79.729397
32	t = 35.000000	I = 0.203691	k = 0.001477	l = 0.125000	F = 80.084130
33	t = 40.000000	I = 0.203297	k = 0.001648	l = 0.125000	F = 79.774146
34	t = 45.000000	I = 0.201812	k = 0.001812	l = 0.125000	F = 78.613014
35	t = 50.000000	I = 0.200000	k = 0.001967	l = 0.125000	F = 77.207781
36	t = 55.000000	I = 0.198033	k = 0.001967	l = 0.125000	F = 75.696307
37	t = 60.000000	I = 0.196065	k = 0.001967	l = 0.125000	F = 74.199774
38	t = 65.000000	I = 0.195082	k = 0.001967	l = 0.125000	F = 73.457111
39	t = 70.000000	I = 0.195082	k = 0.001967	l = 0.125000	F = 73.457111
40	t = 75.000000	I = 0.195082	k = 0.001967	l = 0.125000	F = 73.457111
41	t = 80.000000	I = 0.195082	k = 0.001967	l = 0.125000	F = 73.457111
42	t = 85.000000	I = 0.195082	k = 0.001967	l = 0.125000	F = 73.457111
43	t = 90.000000	I = 0.196065	k = 0.001967	l = 0.125000	F = 74.199774
44	t = 95.000000	I = 0.198033	k = 0.001967	l = 0.125000	F = 75.696307
45	t = 100.000000	I = 0.200000	k = 0.001967	l = 0.125000	F = 77.207781

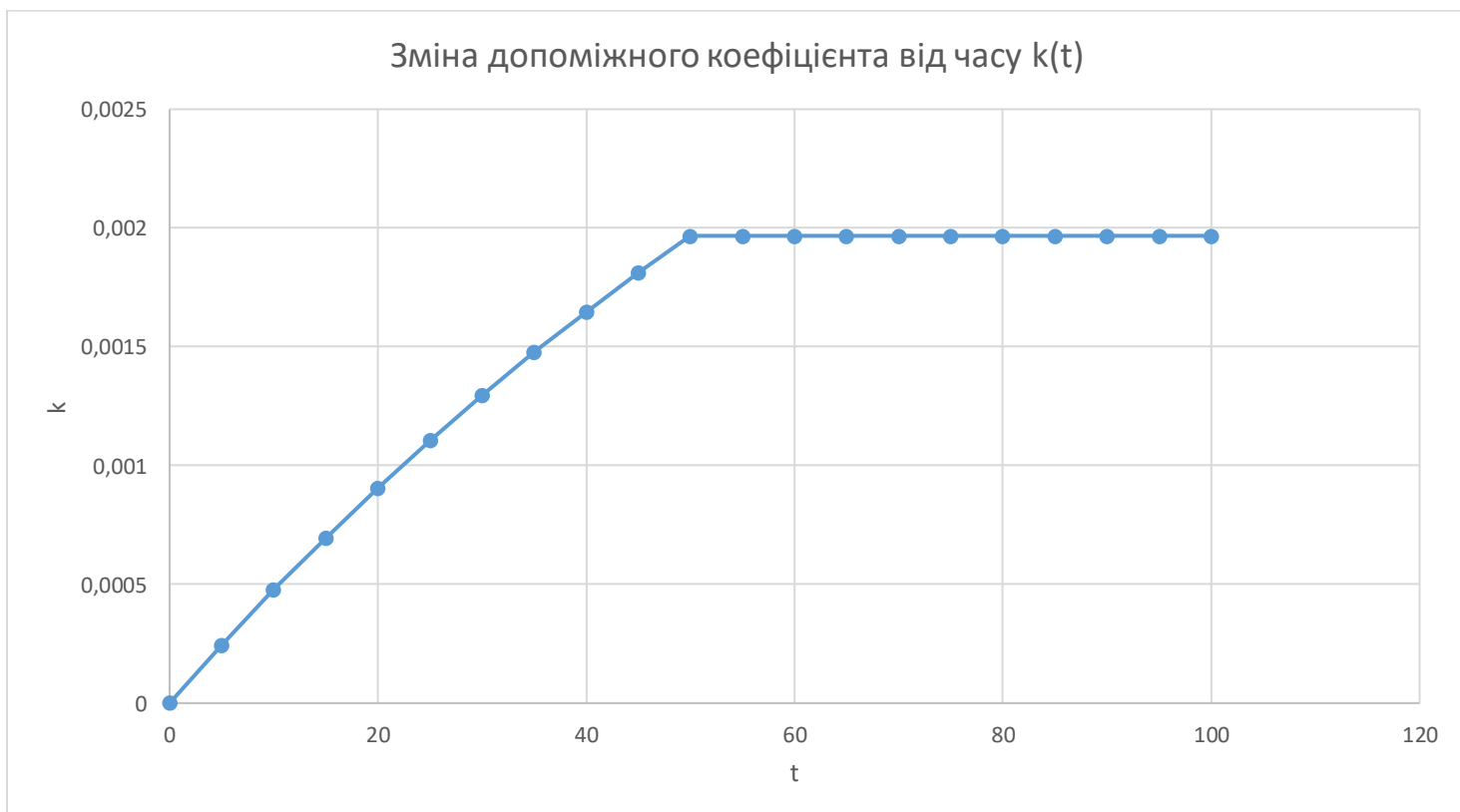
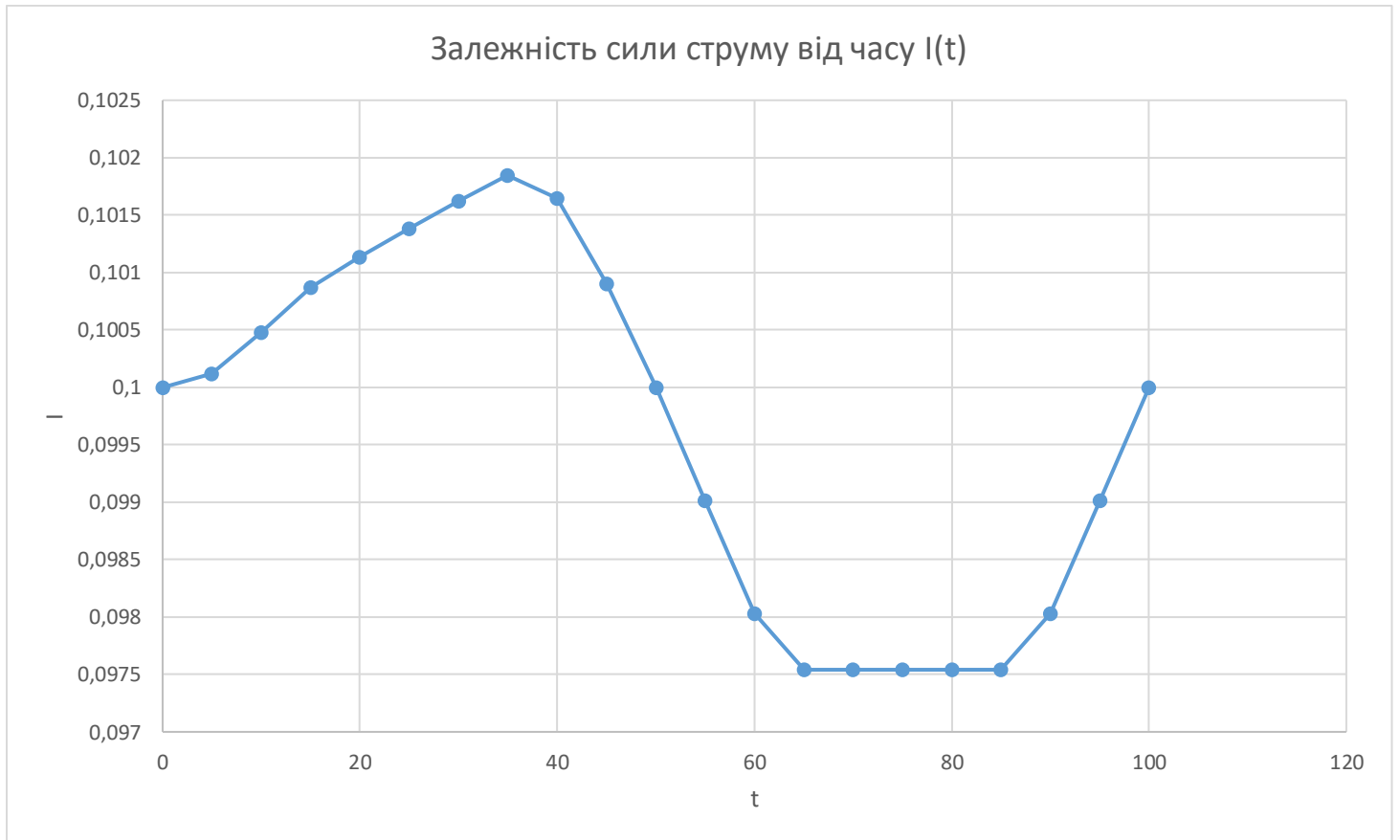
Рисунок 6 - результати роботи програми для варіанта 1 та варіанта 2

46					
47	Variant 3:				
48	t = 0.000000	I = 0.300000	k = 0.000000	l = 0.100000	F = 763.407015
49	t = 5.000000	I = 0.300366	k = 0.000244	l = 0.105000	F = 694.122213
50	t = 10.000000	I = 0.301427	k = 0.000476	l = 0.110000	F = 636.933121
51	t = 15.000000	I = 0.302612	k = 0.000696	l = 0.115000	F = 587.339839
52	t = 20.000000	I = 0.303399	k = 0.000906	l = 0.120000	F = 542.224151
53	t = 25.000000	I = 0.304147	k = 0.001106	l = 0.125000	F = 502.183074
54	t = 30.000000	I = 0.304860	k = 0.001296	l = 0.125000	F = 504.537590
55	t = 35.000000	I = 0.305537	k = 0.001477	l = 0.125000	F = 506.782384
56	t = 40.000000	I = 0.304945	k = 0.001648	l = 0.125000	F = 504.820767
57	t = 45.000000	I = 0.302718	k = 0.001812	l = 0.125000	F = 497.472978
58	t = 50.000000	I = 0.300000	k = 0.001967	l = 0.125000	F = 488.580489
59	t = 55.000000	I = 0.297049	k = 0.001967	l = 0.125000	F = 479.015693
60	t = 60.000000	I = 0.294098	k = 0.001967	l = 0.125000	F = 469.545448
61	t = 65.000000	I = 0.292622	k = 0.001967	l = 0.125000	F = 464.845782
62	t = 70.000000	I = 0.292622	k = 0.001967	l = 0.125000	F = 464.845782
63	t = 75.000000	I = 0.292622	k = 0.001967	l = 0.125000	F = 464.845782
64	t = 80.000000	I = 0.292622	k = 0.001967	l = 0.125000	F = 464.845782
65	t = 85.000000	I = 0.292622	k = 0.001967	l = 0.125000	F = 464.845782
66	t = 90.000000	I = 0.294098	k = 0.001967	l = 0.125000	F = 469.545448
67	t = 95.000000	I = 0.297049	k = 0.001967	l = 0.125000	F = 479.015693
68	t = 100.000000	I = 0.300000	k = 0.001967	l = 0.125000	F = 488.580489

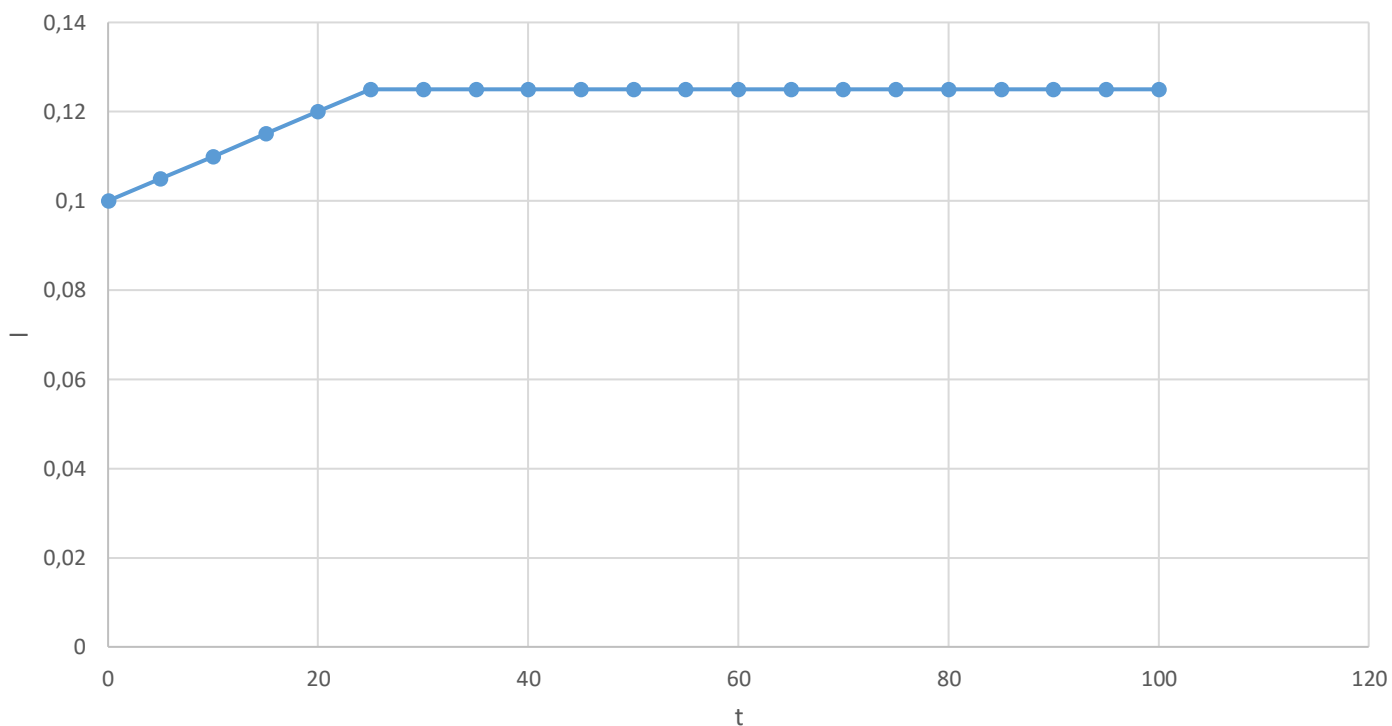
Рисунок 7 - результати роботи програми для варіанта 3

10. Графіки

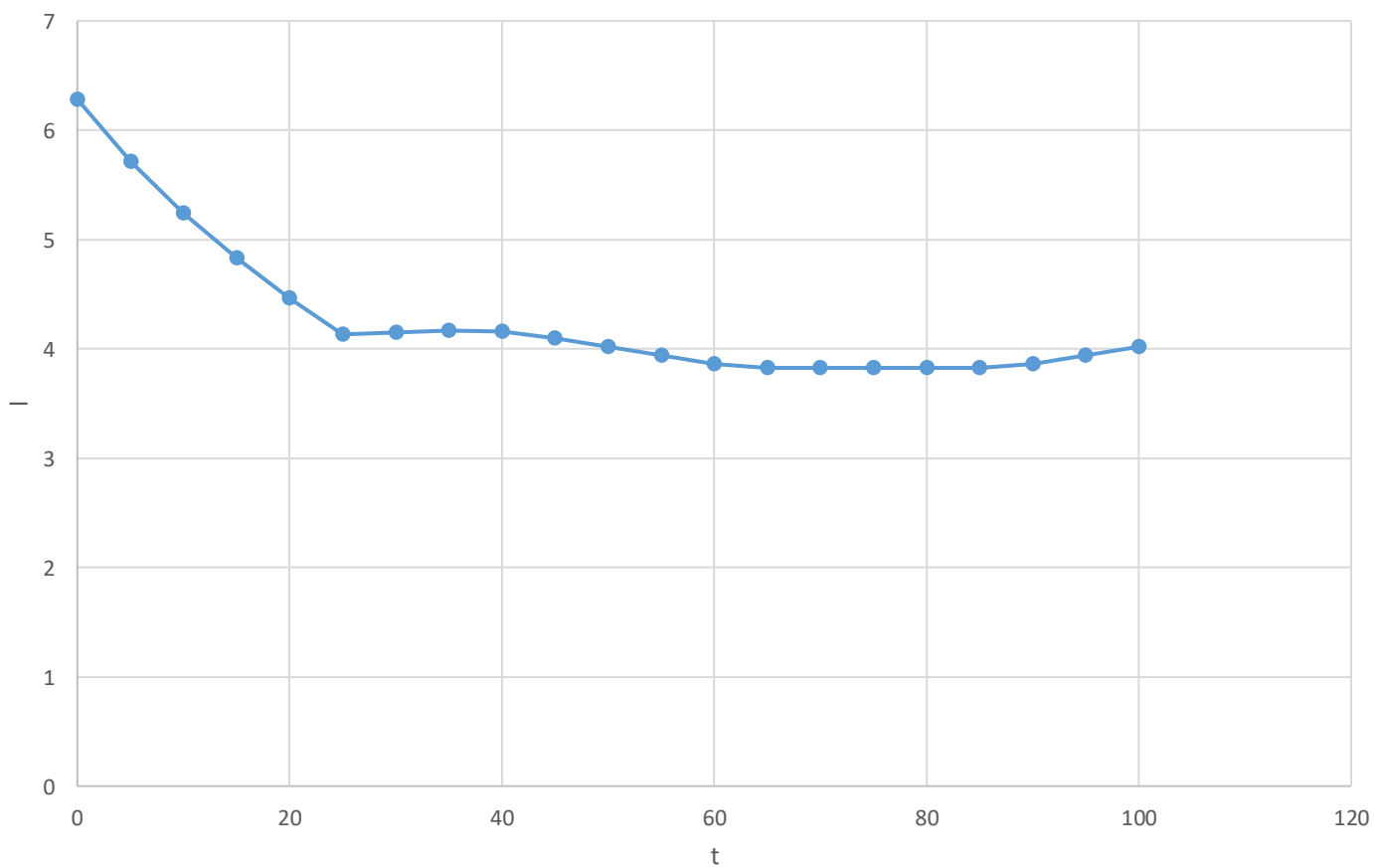
Варіант 1:



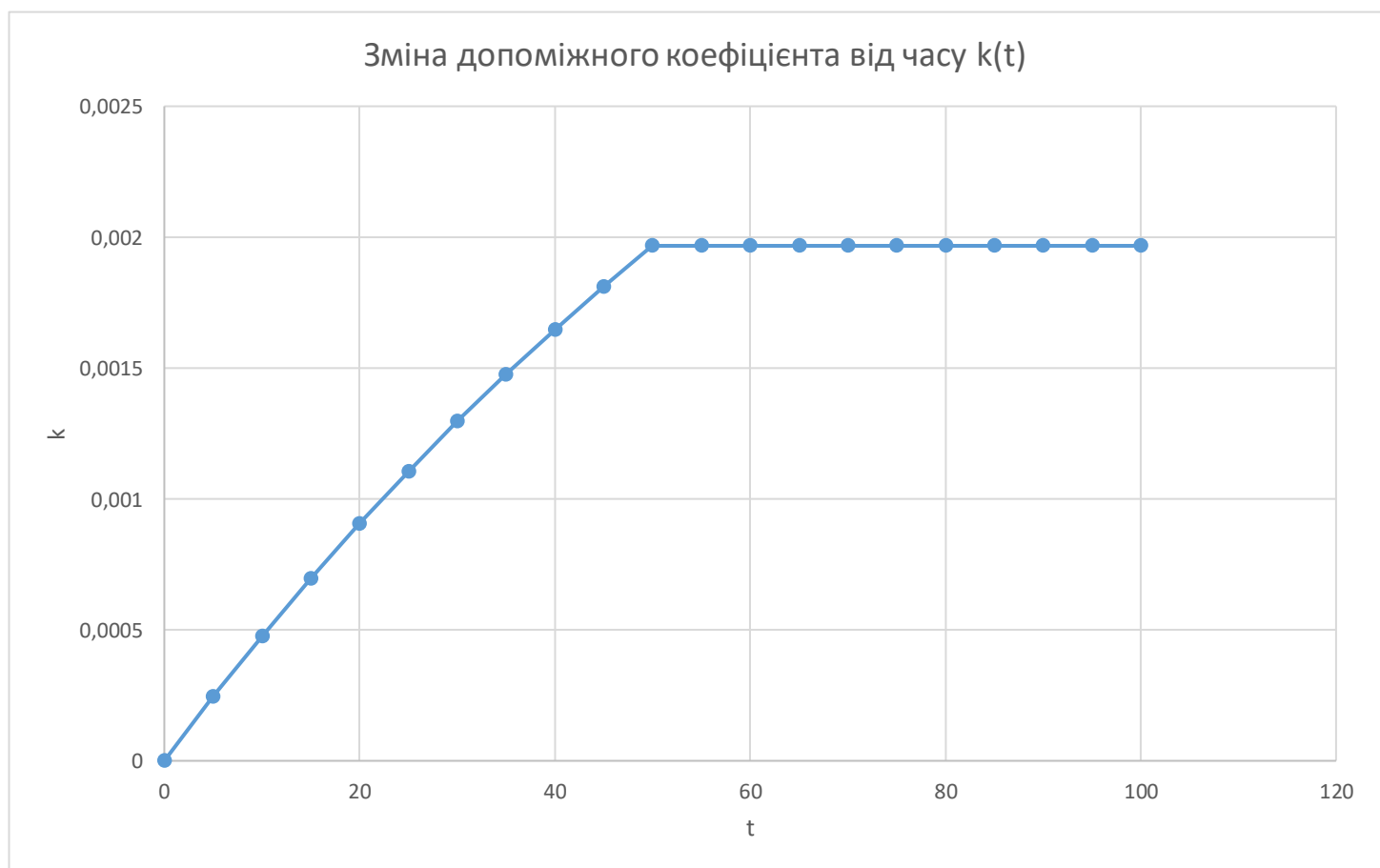
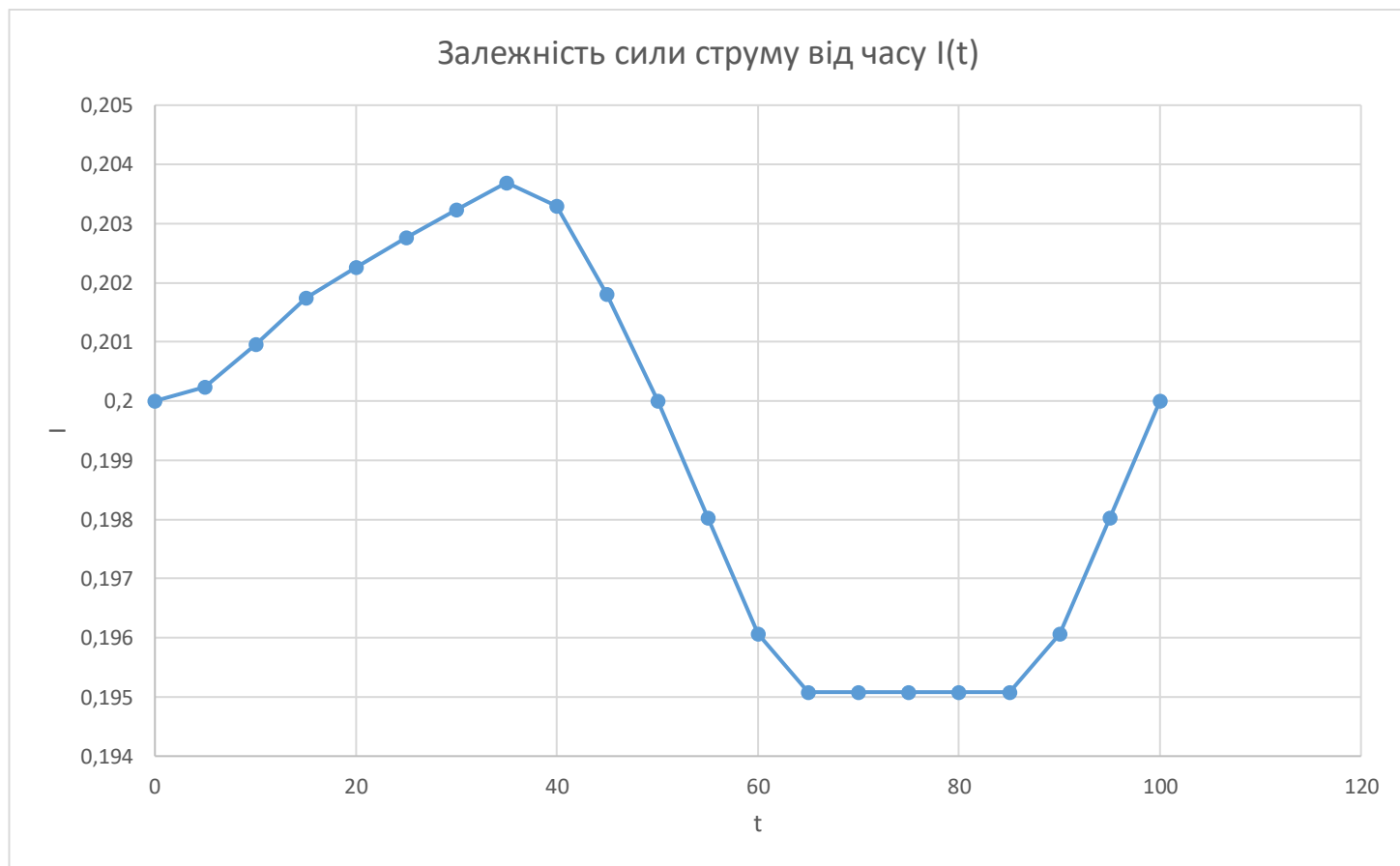
Залежність величини повітряного зазору від часу $l(t)$



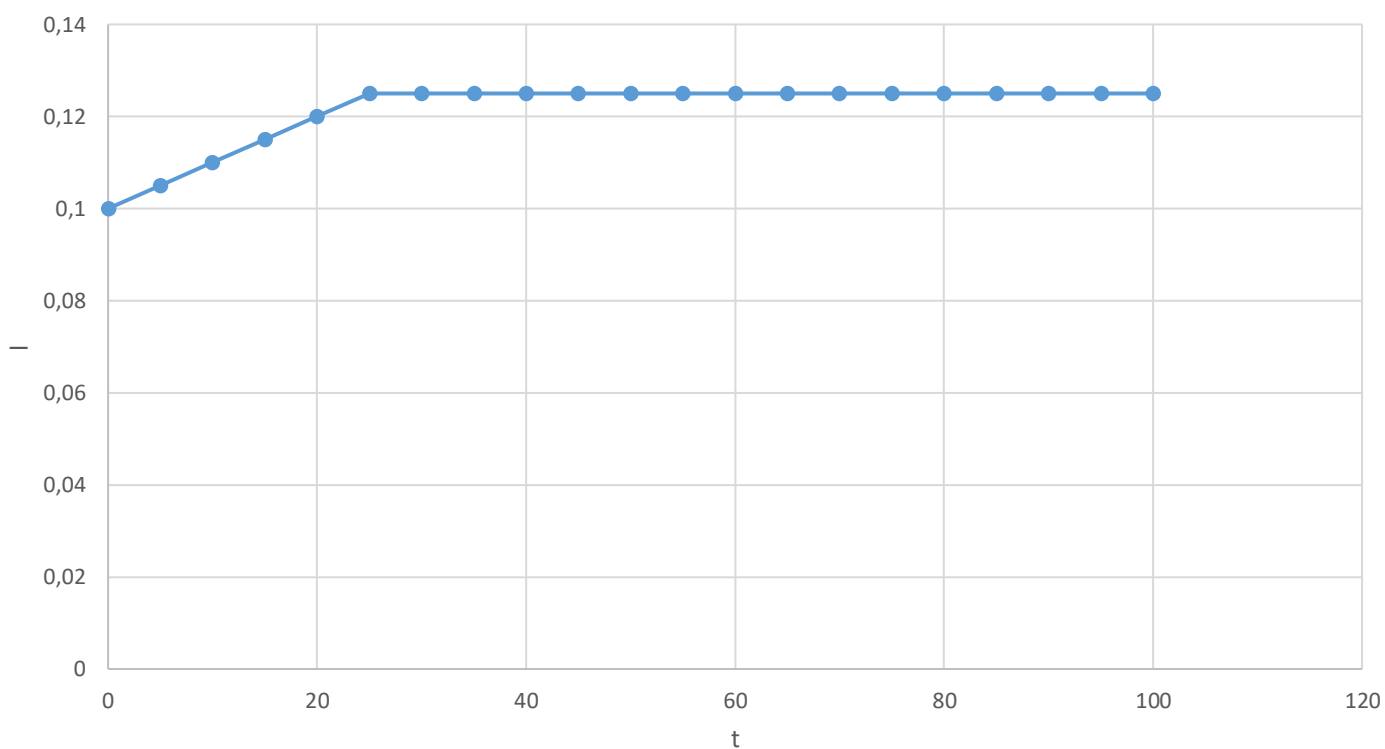
Залежність тягового зусилля від часу $F(t)$



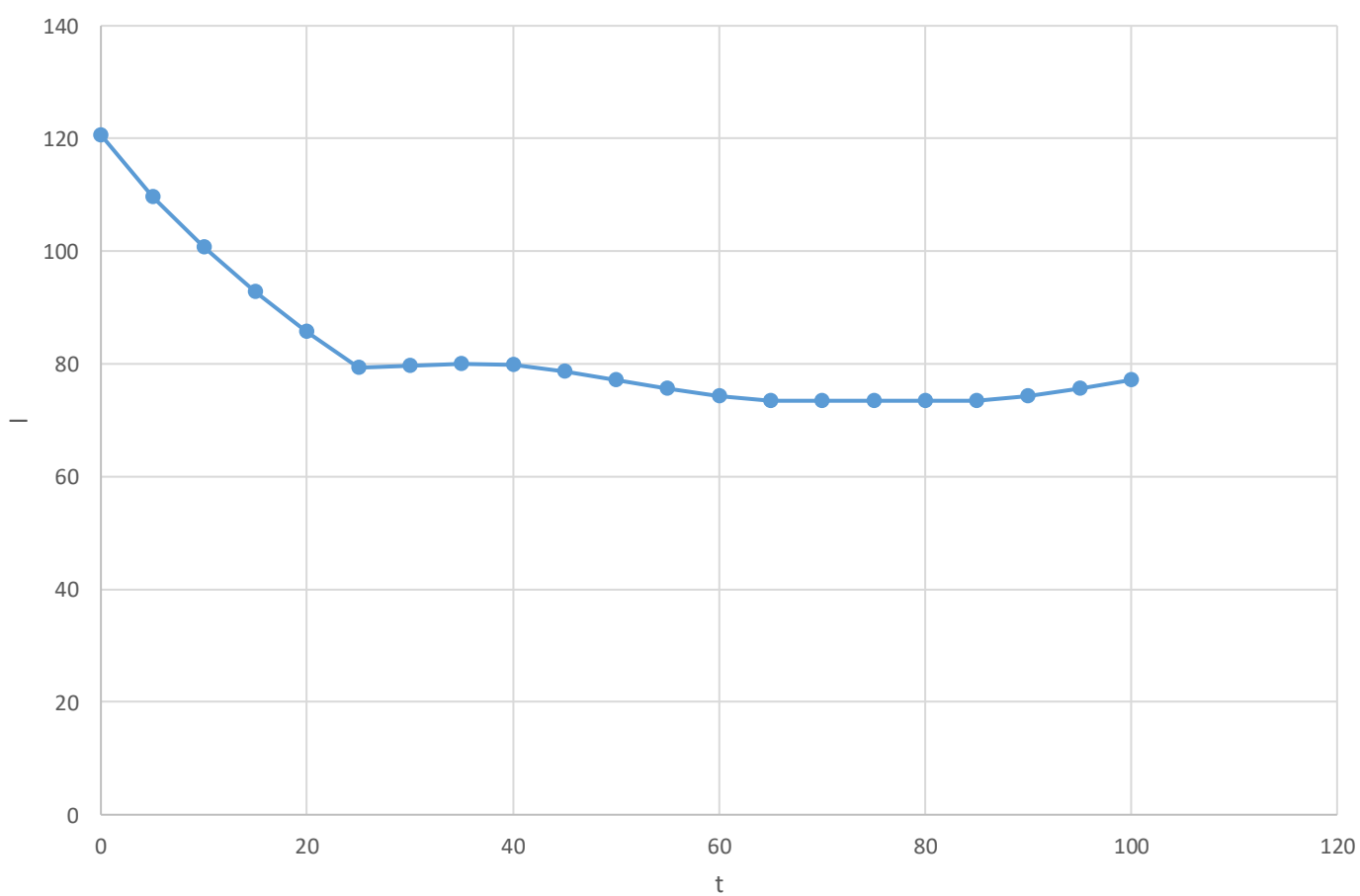
Варіант 2:



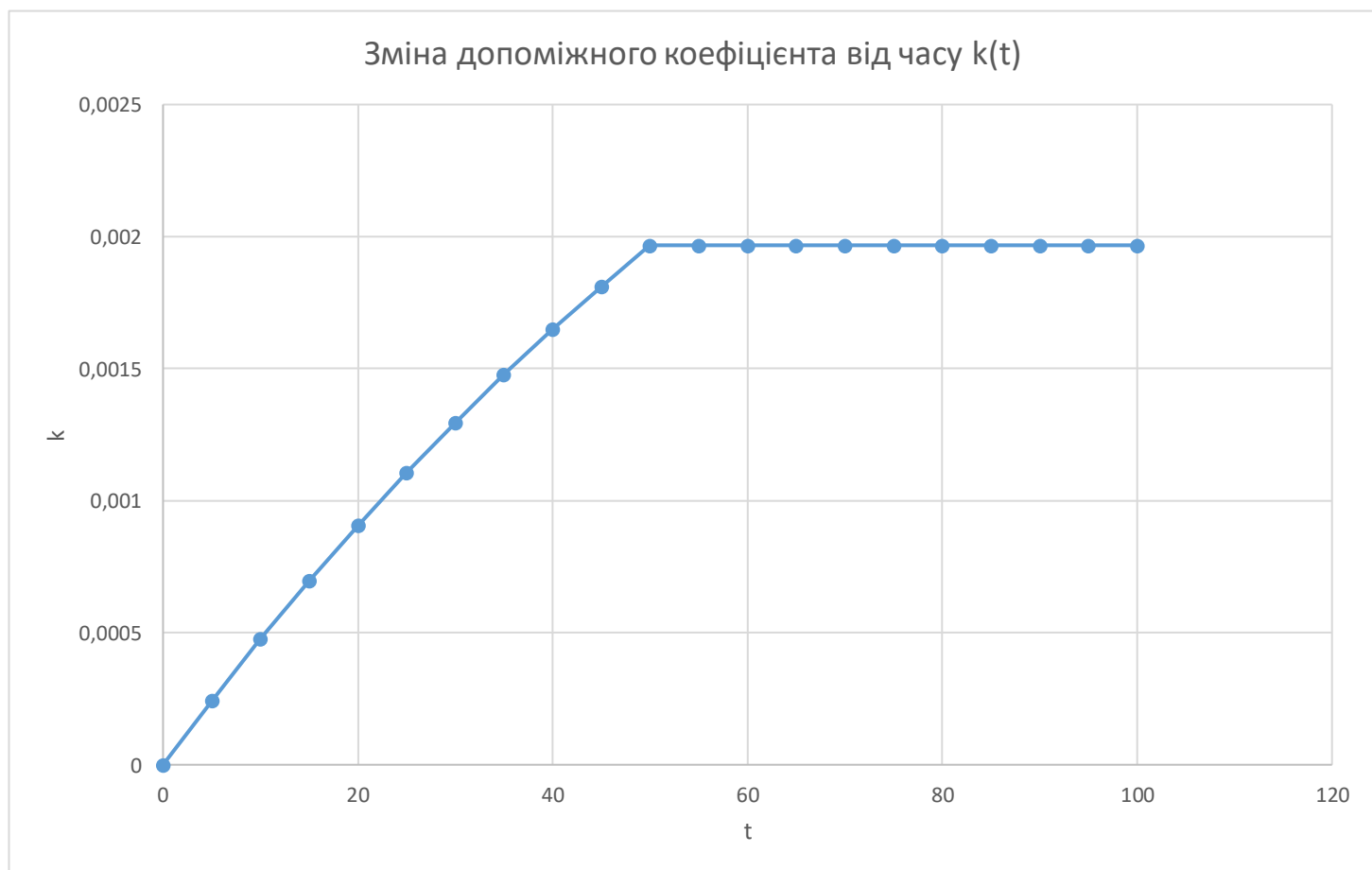
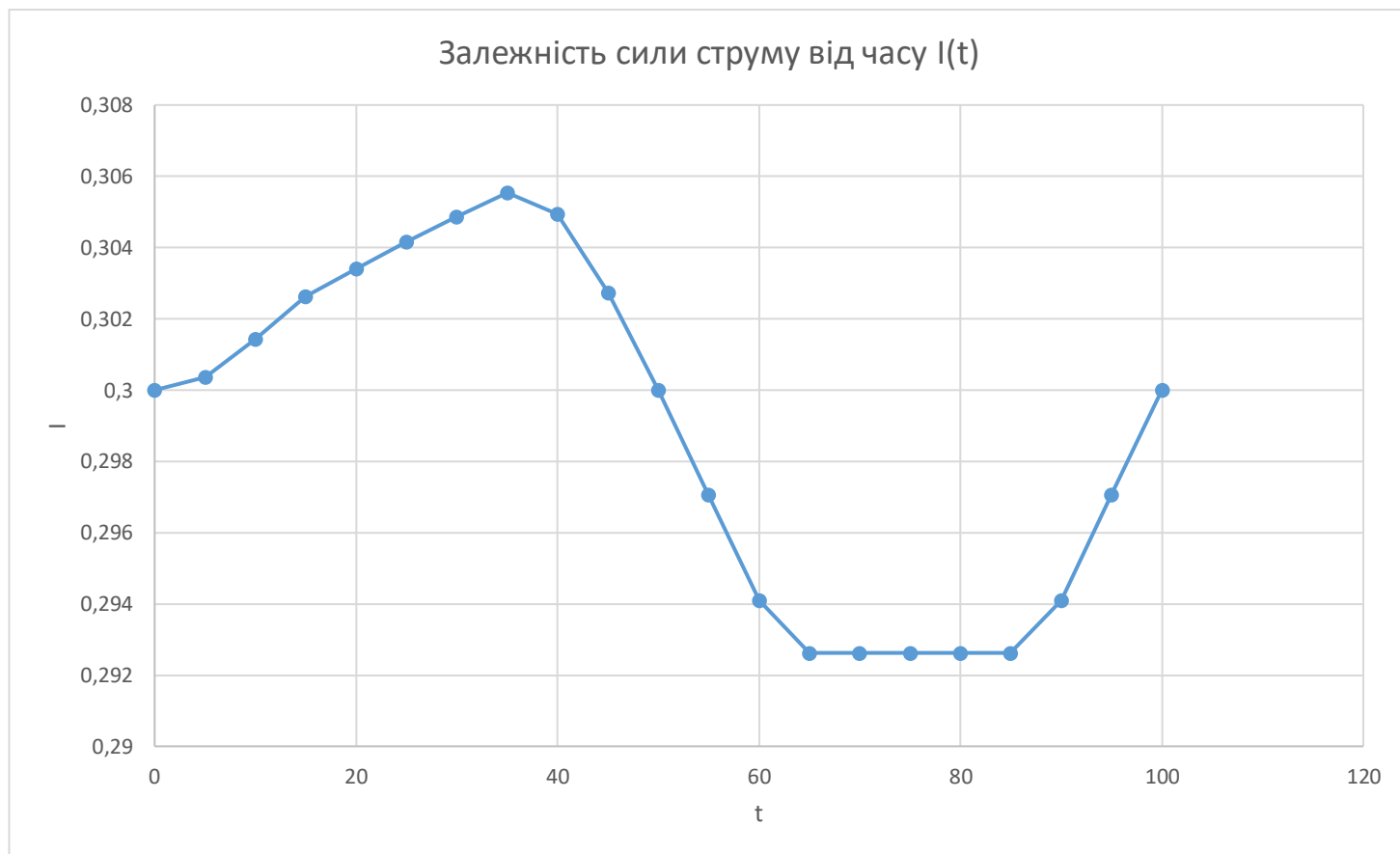
Залежність величини повітряного зазору від часу $l(t)$



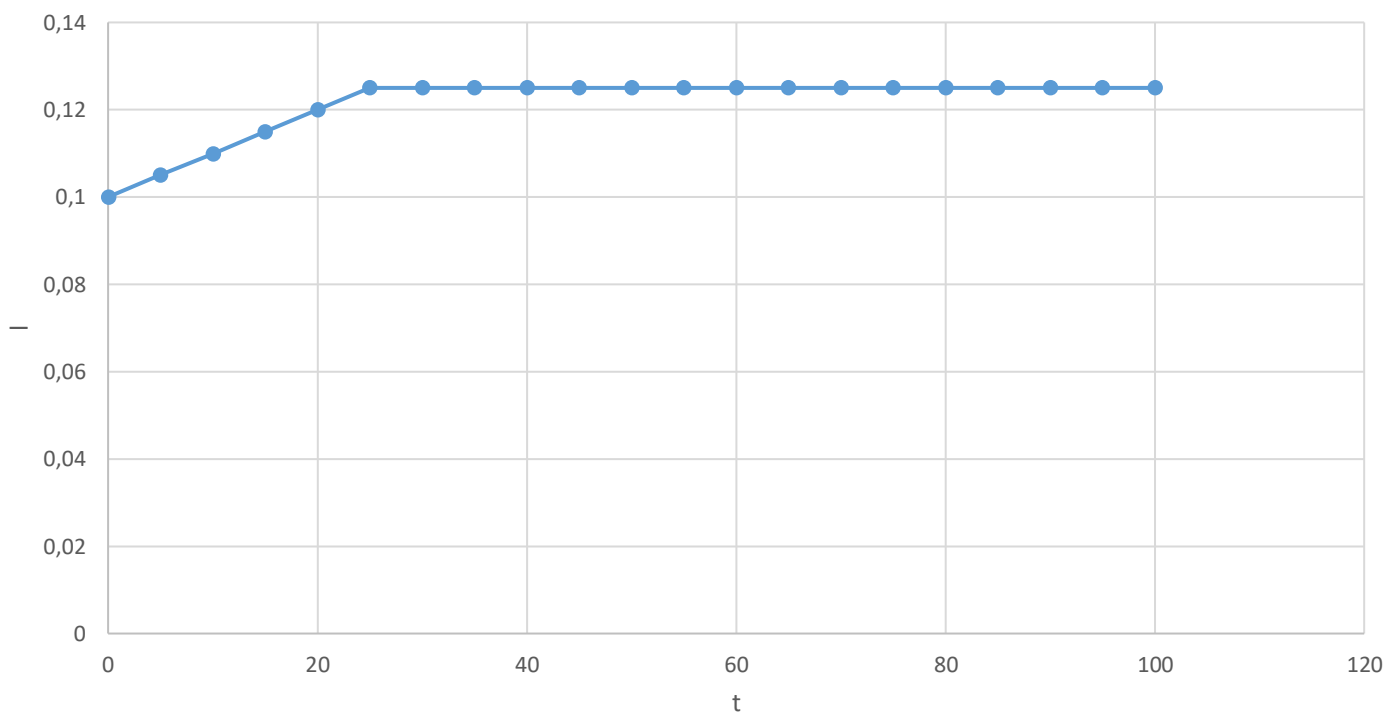
Залежність тягового зусилля від часу $F(t)$



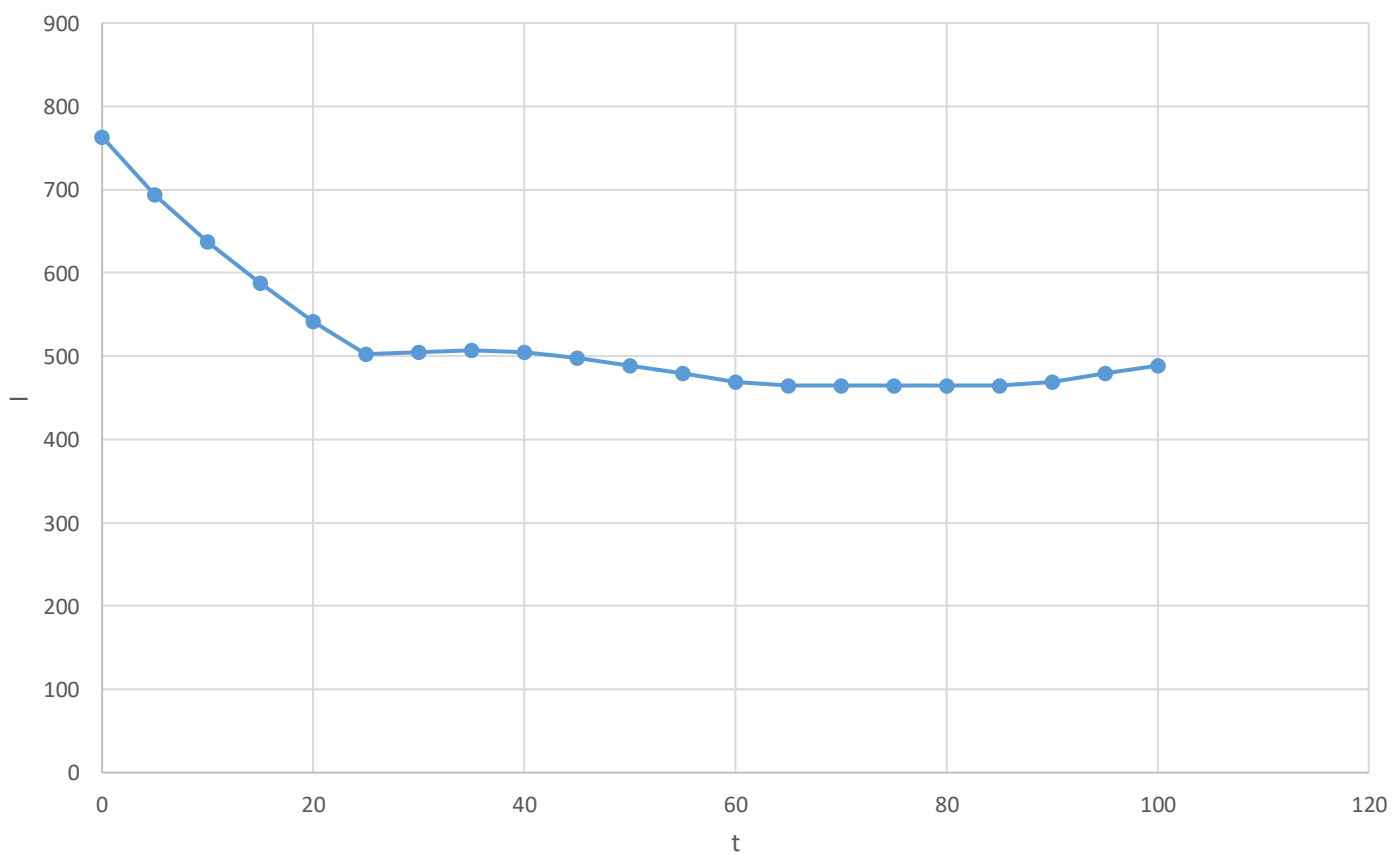
Варіант 3:



Залежність величини повітряного зазору від часу $l(t)$



Залежність тягового зусилля від часу $F(t)$



Висновки

Під час виконання курсової роботи з було розроблено програму для моделювання тягового зусилля електромагніту постійного струму. Програма реалізована на мові програмування C++ у середовищі Microsoft Visual Studio.

У результаті проведеної роботи було досягнуто наступних результатів:

- Створено алгоритм для розрахунку динамічних параметрів системи (струму I , коефіцієнта k , повітряного зазору l , тягової сили F) на основі даних завдання курсової роботи.
- Застосовано механізм динамічного виділення пам'яті `malloc` для створення масиву структур. Після завершення обчислення реалізовано звільнення ресурсів `free`, що запобігає витоку пам'яті.
- Реалізовано автоматичне зчитування 9-ти фізичних параметрів із текстового файлу `InputData.txt` та запис результату програми у файл `OutputData.txt`.
- Описано помилки які програма може мати, для швидкого налагодження та їх вирішення.
- На основі отриманих даних було побудовано графіки часових залежностей у Microsoft Excel, які підтверджують коректність функціонування математичної моделі та дозволяють візуально оцінити динаміку роботи електромагніту.
- Розроблено детальні блок-схеми для головної функції `main` та всіх функцій розрахунків (`find_k`, `find_I`, `find_l`, `find_F`). Це дозволило чітко представити логіку послідовності роботи програми.
- Завдяки використанню `double` та впровадженню супер точності ($1e-9$ та $1e-15$) досягнуто високої точності розрахунків.

Виконання даної курсової роботи стало для мене важливим етапом переходу від школяра до студента. Це був перший досвід, де знання з різних дисциплін нарешті склалися в одне ціле. Я зміг на практиці поєднати проєктування алгоритмів у вигляді блок-схем зі «Вступу до спеціальності», професійне оформлення змісту з

«організації та обробки електронної інформації» та безпосередньо розробку на мові C++.

Найбільше задоволення я отримав від моменту, коли було зроблено перевірку вихідних даних з моєї програми в графіках Excel – це стало доказом того, що код написаний правильно. Але було непросто будувати детальні блок-схеми. Проте саме цей процес навчив мене бачити логіку програми.

Окремим викликом для мене став розподіл часу на таку масштабну роботу. Оскільки проєкт складався з багатьох етапів, я навчився розбивати велику мету на малі щоденні кроки. Такий підхід допоміг уникнути перевантаження та дозволив приділити належну увагу кожному модулю програми.

Список використаної літератури

- C++ Introduction [Електронний ресурс] // W3Schools. — URL: https://www.w3schools.com/cpp/cpp_intro.asp
- Лекція №5. Блок-схеми, їх види та застосування. Інструменти візуалізації алгоритмів [Електронний ресурс] // Платформа MIX СумДУ. — URL: <https://mix.sumdu.edu.ua/textbooks/120744/2871805/index.html>
- Тема 1. Конструкції мови C++. Лекція №1 [Електронний ресурс] // Платформа MIX СумДУ. — URL: <https://mix.sumdu.edu.ua/textbooks/121885/2857372/index.html#p7>
- C Standard Library headers [Електронний ресурс] // cppreference.com. — URL: <https://en.cppreference.com/w/c/header.html>

Додаток А. Програмний код

InputData.txt:

100 5 1000 10 0.1 0.005 0.1 0.01 0.01

100 5 2000 12 0.2 0.005 0.1 0.01 0.01

100 5 3000 15 0.3 0.005 0.1 0.01 0.01

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
struct InputData {
```

```
    double T;
```

```
    double dt;
```

```
    double w;
```

```
    double Sc;
```

```
    double i0;
```

```
    double k0;
```

```
    double l0;
```

```
    double m;
```

```
    double n;
```

```
};
```

```
double find_k(double k0, double m, double t, double T) {
```

```
    if (t >= 0 && t < T / 2.0) {
```

```
        return k0 * (1.0 - exp(-m * t));
```

```
    }
```

```
    else {
```

```
        return k0 * (1.0 - exp(-m * (T / 2.0)));
```

```
    }
```

```
}
```

```
double find_I(double i0, double T, double t, double k) {
```

```

        if (t >= 0 && t < T / 8.0) {
            return i0 * (1.0 + k * t);
        }
        else if (t >= T / 8.0 && t < (3.0 * T) / 8.0) {
            return i0 * (1.0 + k * (T / 8.0));
        }
        else if (t >= (3.0 * T) / 8.0 && t < (5.0 * T) / 8.0) {
            return i0 * (1.0 + k * (T / 8.0) - k * (t - (3.0 * T)
/ 8.0));
        }
        else if (t >= (5.0 * T) / 8.0 && t < (7.0 * T) / 8.0) {
            return i0 * (1.0 - k * (T / 8.0));
        }
        else if (t >= (7.0 * T) / 8.0 && t <= T + 1e-9) {
            return i0 * (1.0 - k * (T / 8.0) + k * (t - (7.0 * T)
/ 8.0));
        }
        return 0;
    }

    double find_l(double l0, double n, double t, double T) {
        if (t >= 0 && t < T / 4.0) {
            return l0 * (1.0 + n * t);
        }
        else {
            return l0 * (1.0 + n * (T / 4.0));
        }
    }

    double find_F(double I, double w, double Sc, double lb) {
        const double PI = 3.14159265358979323846;
        double numerator = 0.00001 * (0.4 * PI * I * w) * (0.4 * PI
* I * w) * Sc;
        double denominator = 8 * PI * lb * lb;
        if (denominator < 1e-15) {

```

```

        printf("Error: denominator = 0");
        return 0;
    }
    return numerator / denominator;
}

int main() {
    int count = 3;
    int i;

    struct InputData* x;
    x = (struct InputData*)malloc(count * sizeof(struct
InputData));
    if (x == NULL) {
        printf("Error: memory");
        return 1;
    }

    FILE* fin;
    fin = fopen("InputData.txt", "r");
    if (fin == NULL) {
        printf("Error: cannot open InputData.txt");
        return 1;
    }

    for (i = 0; i < count; i++) {
        if (fscanf(fin, "%lf %lf %lf %lf %lf %lf %lf %lf %lf",
            &x[i].T, &x[i].dt, &x[i].w, &x[i].Sc, &x[i].i0,
&x[i].k0, &x[i].l0, &x[i].m, &x[i].n) != 9) {
            printf("Error: you need 9 values in %d variant\n", i +
1);

            fclose(fin);
            free(x);
            return 1;

```

```

        }
    }
    fclose(fin);

    FILE* p1;
    p1 = fopen("OutputData.txt", "w");
    if (p1 == NULL) {
        printf("Error: cannot create OutputData.txt");
        free(x);
        return 1;
    }

    for (i = 0; i < count; i++) {
        fprintf(p1, "Variant %d:\n", i + 1);
        printf("Variant %d:\n", i + 1);
        for (double t = 0; t <= x[i].T + (x[i].dt / 2.0); t +=
x[i].dt) {
            double k = find_k(x[i].k0, x[i].m, t, x[i].T);
            double I = find_I(x[i].i0, x[i].T, t, k);
            double l = find_l(x[i].l0, x[i].n, t, x[i].T);
            double F = find_F(I, x[i].w, x[i].Sc, l);

            fprintf(p1, "t = %lf  I = %lf  k = %lf  l = %lf
F = %lf\n", t, I, k, l, F);
            printf("t = %lf  I = %lf  k = %lf  l = %lf  F
= %lf\n", t, I, k, l, F);
        }
        fprintf(p1, "\n");
    }
    fclose(p1);
    free(x);
    return 0;
}

```